

Universidad Tecnológica Nacional.

Tecnicatura Universitaria en Programación. 1Pro1.

7 de junio del 2026.

Programación II.

Docente Roco.

Alumnos Danilo Serrano, Jesús Ramírez

y Lautaro Fernandez.

Trabajo Práctico Integrador: **Juego de Naipes Españoles.**

Integrantes del equipo.

Jesús Ramírez, Lautaro Fernandez, Danilo Serrano.

Identificación del Capitán del Equipo.

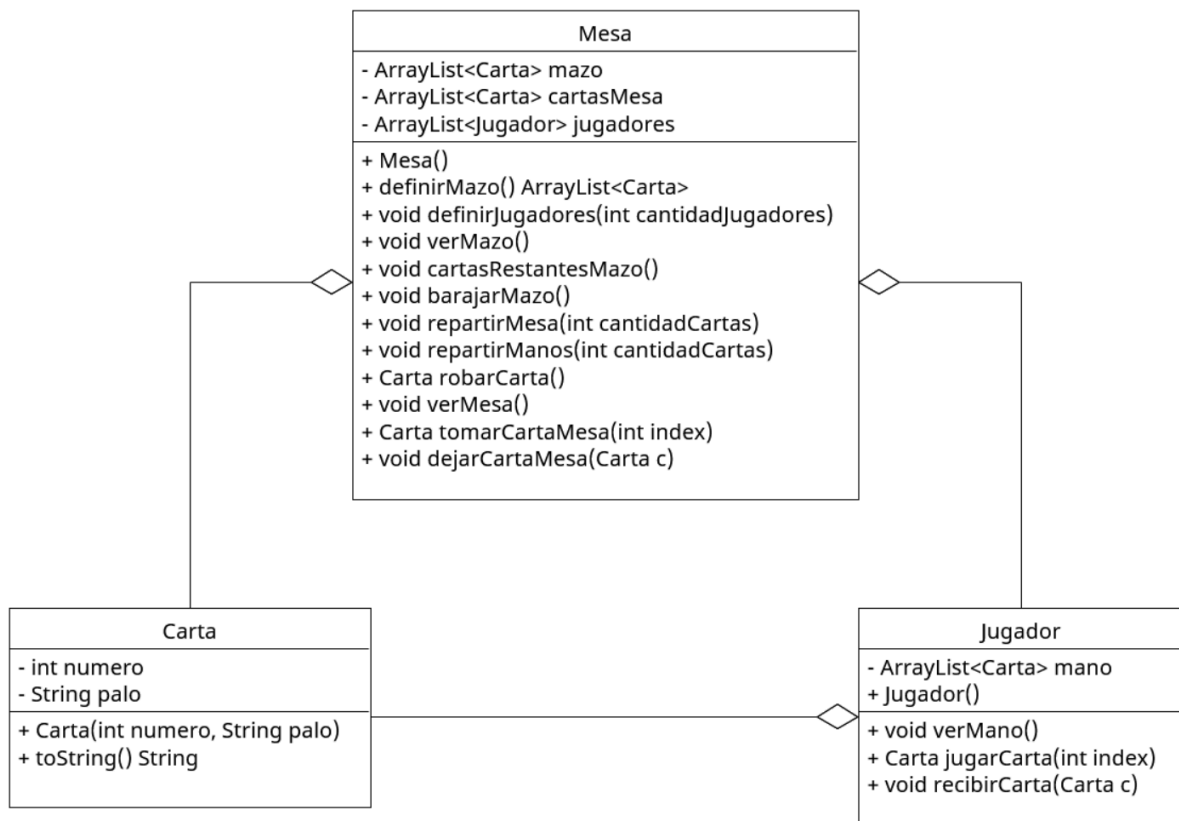
Danilo Serrano.

Análisis del problema.

El objetivo de este proyecto es diseñar e implementar una aplicación en Java que simule un juego con la baraja española tradicional de 40 cartas, debe tener los cuatro palos tradicionales (Oro, Bastos, Copas y Espadas) y los números del 1 al 12, excluyendo los ochos y los nueves. Actualmente, la solución se modela a través de cuatro clases donde la lógica se encuentra fuertemente centralizada en la clase `Mesa`, la cual actúa como motor principal al fabricar el mazo y registrar a los integrantes. Por su parte, la clase `Carta` funciona de manera inmutable almacenando el valor y palo de cada naipes, la clase `Jugador` gestiona las manos de los participantes, y la clase `Main` ejecuta el bucle de turnos mediante un menú interactivo por consola.

Para administrar el constante movimiento de los naipes, se justificó el uso de la estructura dinámica `ArrayList` sobre los arrays, permitiendo que el volumen de cartas en el mazo, la mesa y las manos de los jugadores cambie en tiempo real sin generar índices vacíos en la memoria. El diseño optimiza el rendimiento del código al utilizar funciones nativas como `Collections.shuffle()` para el mezclado y el método `removeLast()` para eliminar el último elemento. Hasta el momento tenemos un diseño basado puramente en la programación orientada a objetos, redistribuyendo las tareas entre `Mesa` y `Mazo` para descentralizar el código y lograr la cohesión, el encapsulamiento y la correcta delegación de responsabilidades de forma simple y natural.

Diagrama UML.



Justificación de las decisiones de diseño.

La primera decisión clave fue la estructuración del sistema en clases independientes, buscando aplicar el principio de alta cohesión al separar claramente las tareas de Carta, Jugador, Mesa y Mazo. Como se refleja en el Diagrama UML, el modelo está planteado conceptualmente para imitar las interacciones de una partida real, donde cada entidad del dominio tiene un rol asignado. Aunque el proyecto se encuentra en una etapa intermedia donde la clase **Mesa** centraliza la lógica operativa, la arquitectura visualizada en el diagrama facilita que la gestión de la baraja se delegue de forma progresiva y natural a la clase **Mazo**. Así mismo, se priorizó el encapsulamiento al definir los atributos de la carta como privados, garantizando que su valor y palo permanezcan inmutables durante todo el ciclo de vida del juego, mientras que las relaciones de asociación en el UML exponen cómo fluyen los naipes entre los participantes sin perder la consistencia de los datos.

Por otro lado, se justificó técnicamente la elección de `ArrayList` como la estructura de datos central frente a los arreglos fijos tradicionales, ya que su naturaleza dinámica permite que las colecciones representadas en el diagrama cambien de tamaño en tiempo real sin generar desbordamientos de memoria ni dejar índices vacíos. El diseño optimiza el rendimiento general del código al utilizar la utilidad nativa `Collections.shuffle()` para resolver el mezclado en una sola línea,

y al implementar el método `removeLast()` para la extracción de naipes. Al retirar siempre la última carta de la lista en lugar de la primera, se evitó que Java tuviera que reordenar internamente las posiciones de los elementos restantes en cada turno, logrando así una simulación eficiente, directa y limpia que se alinea con las buenas prácticas de la programación orientada a objetos sin necesidad de incorporar la complejidad de un patrón de diseño.

Análisis sobre posibles patrones de diseño aplicables.

Tras evaluar los patrones de diseño para este proyecto, determinamos que la arquitectura del sistema no justifica incorporar estructuras complejas. El criterio principal fue evitar que un patrón de diseño, en este proyecto, empeore el desarrollo del software. Al tratarse de una simulación a pequeña escala con un mazo fijo de 40 naipes inmutables, introducir patrones tradicionales aumentaría la complejidad del código y del diagrama UML sin aportar un valor real ni optimizar la resolución del problema.

Por este motivo se descartó Factory Method, ya que todas las cartas pertenecen a un único tipo básico (Carta) sin variantes complejas que requieran una fábrica, e igualmente se rechazó Singleton porque, aunque se use un solo mazo, este patrón acoplaría el código de forma global y bloquearía la flexibilidad futura de simular partidas con múltiples mazos o mesas en paralelo. Para el alcance actual, las herramientas puras de la programación orientada a objetos bastan para garantizar una solución eficiente y prolija.

A nivel de estructura, el código actual funciona de forma muy parecida al patrón Facade, ya que la clase Mesa actúa como un intermediario, simplifica el juego y evita que el Main tenga que mezclarse con la lógica interna de los ArrayList o los índices de las cartas. El Main solo pide acciones sencillas y la Mesa se encarga de resolver todo el trabajo por detrás.

Por otro lado, si en el futuro quisiéramos cambiar las reglas del juego, el único patrón que serviría sería Strategy (Estrategia). Este patrón se aplicaría para separar la forma de barajar. Así, podríamos elegir en el momento si mezclamos de forma automática con `Collections.shuffle()` o si simulamos un corte de mazo manual, permitiendo agregar nuevas formas de mezclar más adelante sin tener que modificar el código que ya funciona.

Capturas del programa en ejecución.

```
---Turno del Jugador 1.  
1. Ver mesa  
2. Ver mano  
3. Robar de la mesa  
4. Robar del mazo  
5. Dejar en la mesa  
6. Ver mazo  
7. Cerrar programa  
Op:
```

```
1. Ver mesa  
2. Ver mano  
3. Robar de la mesa  
4. Robar del mazo  
5. Dejar en la mesa  
6. Ver mazo  
7. Cerrar programa  
Op: 6
```

```
---Cartas del Mazo.  
7 de Espadas  
11 de Oro  
6 de Bastos  
2 de Bastos  
10 de Copas  
2 de Oro  
10 de Espadas
```

```
---Turno del Jugador 1.  
1. Ver mesa  
2. Ver mano  
3. Robar de la mesa  
4. Robar del mazo  
5. Dejar en la mesa  
6. Ver mazo  
7. Cerrar programa  
Op: 1
```

```
---Cartas en la Mesa.  
0. 5 de Oro  
1. 1 de Copas  
2. 4 de Copas  
3. 2 de Copas
```

```
1. Ver mesa  
2. Ver mano  
3. Robar de la mesa  
4. Robar del mazo  
5. Dejar en la mesa  
6. Ver mazo  
7. Cerrar programa  
Op: 2  
---Mano.  
0. 3 de Copas  
1. 6 de Copas  
2. 5 de Copas
```

```
1. Ver mesa
2. Ver mano
3. Robar de la mesa
4. Robar del mazo
5. Dejar en la mesa
6. Ver mazo
7. Cerrar programa
Op: 4
Robaste: 7 de Bastos
```

---Turno del Jugador 2.

```
1. Ver mesa
2. Ver mano
3. Robar de la mesa
4. Robar del mazo
5. Dejar en la mesa
6. Ver mazo
7. Cerrar programa
```

Op: 5

---Mano.

```
0. 2 de Espadas
1. 11 de Espadas
2. 6 de Oro
```

Índice de la carta a dejar: 2

Dejaste: 6 de Oro

---Cartas en la Mesa.

```
0. 5 de Oro
1. 1 de Copas
2. 4 de Copas
3. 2 de Copas
4. 6 de Oro
```